```
import random

# User input
demand = int(input("Enter customer demand: "))
current_stock = int(input("Enter current stock: "))

n = int(input("Enter number of warehouses: "))

warehouse_capacity = []
storage_cost = []

for i in range(n):
    print("\nWarehouse", i + 1)
    warehouse_capacity.append(
        int(input("Enter warehouse capacity: "))
    )
    storage_cost.append(
        int(input("Enter storage cost per unit: "))
    )

replenishment_cost = int(input("Enter replenishment cost per unit: "))

POPULATION_SIZE = 10
GENERATIONS = 50
MUTATION_RATE = 0.1


# Generate a random solution
def create_solution():
    return [
        random.randint(0, cap)
        for cap in warehouse_capacity
    ]


# Calculate total cost
def fitness(solution):
    total_stock = current_stock + sum(solution)
    shortage = max(0, demand - total_stock)

    cost = sum(
        solution[i] * (replenishment_cost + storage_cost[i])
        for i in range(n)
    )

    cost += shortage * 1000
    return cost


# Select the best solution from random candidates
def selection(population):
    candidates = random.sample(
        population, min(3, len(population))
```

```
    )
    return min(candidates, key=fitness)


# Crossover
def crossover(parent1, parent2):
    point = random.randint(1, n) if n > 1 else 1

    child = (
        parent1[:point] + parent2[point:]
    )
    return child


# Mutation
def mutation(solution):
    if random.random() < MUTATION_RATE:
        i = random.randint(0, n - 1)
        solution[i] = random.randint(
            0, warehouse_capacity[i]
        )
    return solution


# Genetic Algorithm
def genetic_algorithm():
    population = [
        create_solution()
        for _ in range(POPULATION_SIZE)
    ]

    for _ in range(GENERATIONS):
        new_population = []

        for _ in range(POPULATION_SIZE):
            parent1 = selection(population)
            parent2 = selection(population)

            child = crossover(parent1, parent2)
            child = mutation(child)

            new_population.append(child)

        population = new_population

    return min(population, key=fitness)


# Main program
best = genetic_algorithm()

print("\n--- Optimization Results ---")
print("Customer Demand:", demand)
print("Current Stock:", current_stock)
```

```
for i in range(n):
    print(
        "Warehouse", i + 1,
        "Replenishment:", best[i],
        "Capacity:", warehouse_capacity[i]
    )

print("Total Available Stock:", current_stock + sum(best))

if current_stock + sum(best) >= demand:
    print("Demand Status: Fully Met")
else:
    print("Demand Status: Not Fully Met")

print("Minimum Total Cost:", fitness(best))
```

```
Enter customer demand: 130
Enter current stock: 50
Enter number of warehouses: 3

Warehouse 1
Enter warehouse capacity: 12
Enter storage cost per unit: 6

Warehouse 2
Enter warehouse capacity: 5
Enter storage cost per unit: 20

Warehouse 3
Enter warehouse capacity: 59
Enter storage cost per unit: 13
Enter replenishment cost per unit: 3

--- Optimization Results ---
Customer Demand: 130
Current Stock: 50
Warehouse 1 Replenishment: 12 Capacity: 12
Warehouse 2 Replenishment: 5 Capacity: 5
Warehouse 3 Replenishment: 54 Capacity: 59
Total Available Stock: 121
Demand Status: Not Fully Met
Minimum Total Cost: 10087
```